Assignment 5: Quicksort Algorithm – Implementation, Analysis, and Randomization

Overview

Quicksort is basically one of the most efficient comparison-based sorting algorithms out there, and it's super popular in computer science mainly because of its awesome average-case performance. It works on the classic divide-and-conquer approach we just pick a pivot element, partition the array into smaller subarrays around that pivot, and then recursively sort those parts.

Now, while Quicksort gives us an average-case time complexity of O(n log n) picking a bad pivot can easily mess things up and hit the worst-case complexity of (n^2).

In this assignment, we're implementing both the **deterministic** and **randomized** versions of Quicksort, analyzing their theoretical performance, and comparing their actual runtimes across different input distributions like sorted, reverse-sorted, and random arrays. The main goal here is to get a clear picture of how much pivot selection impacts the algorithm's performance and why randomized Quicksort ends up being way more reliable in practical scenarios.

1. Implementation

Deterministic Quicksort

In this version, we select the last element of the array as the pivot. The array is then partitioned so that elements smaller than the pivot are placed on the left side, while elements greater than or equal to the pivot are placed on the right side. The algorithm recursively sorts both partitions until the entire array is sorted.

Deterministic Quicksort Implementation

[2]:
```
def deterministic_quicksort(arr, low, high):

    if low < high: #Pivot is selected as the last element.
        pivot_index = partition(arr, low, high)
        # Sorting left partition
        deterministic_quicksort(arr, low, pivot_index - 1)
        # Sorting right partition
        deterministic_quicksort(arr, pivot_index + 1, high)
def partition(arr, low, high): #Partition function for Quicksort which places pivot in its correct position.

    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    # placing pivot at correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

Output:

```
#Testing Deterministic Quicksort

numbers = [34, 7, 23, 32, 5, 62]

print("Before sorting:")
print(numbers)

deterministic_quicksort(numbers, 0, len(numbers)-1)

print("\n After sorting:")
print(numbers)
```

```
Before sorting:
[34, 7, 23, 32, 5, 62]

 After sorting:
[5, 7, 23, 32, 34, 62]
```

Conclusion: Deterministic Quicksort Implementation

The deterministic Quicksort code successfully sorted the given input array using the standard divide-and-conquer strategy. The algorithm correctly picked the pivot, partitioned the array into smaller subarrays, and recursively sorted all the parts.

From the output, we can confirm that the implementation is working properly and generating the correct sorted array. Since this version relies on a fixed pivot position, its performance depends heavily on how the input data is arranged. If the partitions stay balanced, we get a solid average-case runtime of O(n log n). But if the chosen pivot keeps giving us totally lopsided splits like with pre-sorted or reverse-sorted inputs then the runtime can easily degrade to the worst-case of O(n^2).

Also, because the implementation uses in-place partitioning, it doesn't waste extra memory on duplicate arrays and only needs about O(log n) auxiliary space on average for the recursive call stack.

Randomized Quicksort Implementation

```
def randomized_quicksort(arr, low, high):
    #Pivot is randomly selected.
    if low < high:
        pivot_index = randomized_partition(arr, low, high)
        randomized_quicksort(arr, low, pivot_index - 1)
        randomized_quicksort(arr, pivot_index + 1, high)

def randomized_partition(arr, low, high):
    # Select random pivot index
    random_index = random.randint(low, high)

    # Move random pivot to end
    arr[random_index], arr[high] = arr[high], arr[random_index]

    return partition(arr, low, high)
```

Output:

```
# Testing Randomized Quicksort

numbers = [34, 7, 23, 32, 5, 62]

print("Before sorting:")
print(numbers)

randomized_quicksort(numbers, 0, len(numbers)-1)

print("\n After sorting:")
print(numbers)
```

```
Before sorting:
[34, 7, 23, 32, 5, 62]

 After sorting:
[5, 7, 23, 32, 34, 62]
```

Conclusion: Randomized Quicksort Implementation

The randomized Quicksort implementation successfully sorted the input array by picking a random pivot at every partitioning step. As we can see from the output, it gives the exact same correct sorted array as the deterministic version.

The biggest advantage here is that picking the pivot randomly makes the algorithm completely independent of the initial array order. This basically eliminates the chance of repeatedly getting bad pivots and ending up with lopsided splits.

Even though the absolute worst-case time complexity on paper is still O(n^2), the odds of hitting it in real-world scenarios are almost zero. In practice, randomized Quicksort gives way more consistent performance and comfortably maintains an expected runtime of O(n log n) no matter what dataset we throw at it.

This is exactly why randomized Quicksort is the safer and more reliable option when we have no idea how our input data is going to look.

2. Performance Analysis

Best-Case Time Complexity

The best case happens when our pivot splits the array into two almost equal halves every single time. At each level of recursion:

- Partitioning takes O(n) time.
- The depth of the recursion tree is roughly log2 n

The recurrence relation looks like this:

$$T(n) = 2T(n/2) + O(n)$$

By applying the Master Theorem, we get:

$$T(n) = O\ (n \log n)$$

So, the best-case time complexity comes out to O(n log n).

Average-Case Time Complexity

In the average case, we assume the pivots give us reasonably balanced splits across calls. They don't have to be perfectly equal, but as long as they aren't totally lopsided, the tree depth stays close to log n. Since each level does O(n) work overall:

$$O(n) * O(\log n) = O(n \log n)$$

This expected time of O(n log n) is precisely why Quicksort is so preferred in real-world applications.

Worst-Case Time Complexity

The worst-case kicks in when our pivot turns out to be either the absolute smallest or largest element every time. This usually happens with:

- Already sorted arrays
- Reverse-sorted arrays
- Inputs where pivot selection keeps failing badly

The recurrence turns into: $T(n) = T(n - 1) + O(n)$

Expanding this summation: $n + (n-1) + (n-2) + \ldots + 1 = n(n+1)/2$

So, $T(n) = O(n^2)$ The recursion depth also spikes to n, giving us the slowest execution possible.

Space Complexity

Since Quicksort works recursively, extra memory is used by the call stack:

- **Best & Average Case:** Stack depth is balanced, so space complexity is O(log n).
- **Worst Case:** Stack depth hits maximum, taking O(n) space.

Note: The array itself is sorted in-place, so the extra space overhead is solely due to recursive stack frames.

3. Randomized Quicksort Analysis

Randomized Quicksort works the exact same way, except for how we pick the pivot. Instead of sticking to a fixed position like the first element, we pick an element randomly from the subarray.

Since pivot choice no longer depends on the initial ordering of the array, the odds of getting terrible splits again become practically zero.

- **Key Advantages:**
  - Consistent O(n log n) expected runtime regardless of initial order.
  - Completely avoids the sorted array trap that breaks deterministic Quicksort.
  - Even though the theoretical worst case is still $O(n^2)$, encountering it in practice is extremely rare.

4. Experimental Observations

- **Random Arrays:** Both versions gave nearly identical runtimes. Random data inherently produces balanced splits, so both hit that sweet O(n log n) mark easily.
- **Sorted Arrays:** The deterministic version took a huge hit here because picking the first element repeatedly created completely lopsided partitions, dragging performance down

towards O(n^2). On the other hand, Randomized Quicksort handled it smoothly without any slowdown.

- **Reverse-Sorted Arrays:** Same story here deterministic Quicksort crashed into its worst-case performance, while Randomized Quicksort stayed fast and efficient.

5. Empirical Analysis

```
sizes = [1000, 5000, 10000]

results = []

for size in sizes:

    datasets = {
        "Random":
        [random.randint(1,100000) for _ in range(size)],

        "Sorted":
        list(range(size)),

        "Reverse Sorted":
        list(range(size,0,-1)),

        "Duplicates":
        [random.randint(1,100) for _ in range(size)]
    }

    for name,data in datasets.items():
        # Deterministic Quicksort
        arr = data.copy()
        start = time.perf_counter()
        deterministic_quicksort(arr,0,len(arr)-1)
        deterministic_time = time.perf_counter()-start

        # Randomized Quicksort
        arr = data.copy()
        start = time.perf_counter()
        randomized_quicksort(arr,0,len(arr)-1)
        randomized_time = time.perf_counter()-start

        results.append([
            size,
            name,
            deterministic_time,
            randomized_time
        ])
```

Output:

	Input Size	Dataset	Deterministic Quicksort Time	Randomized Quicksort Time
0	1000	Random	0.003282	0.004403
1	1000	Sorted	0.069872	0.001306
2	1000	Reverse Sorted	0.029655	0.001225
3	1000	Duplicates	0.001231	0.001596
4	5000	Random	0.005530	0.007580
5	5000	Sorted	1.108559	0.006781
6	5000	Reverse Sorted	0.738593	0.007526
7	5000	Duplicates	0.015147	0.017637
8	10000	Random	0.011825	0.016343
9	10000	Sorted	4.432918	0.015513
10	10000	Reverse Sorted	2.941778	0.015731
11	10000	Duplicates	0.053331	0.059187

Overall Conclusion

The practical results match our theoretical analysis perfectly. Deterministic Quicksort works great on random inputs but fails miserably on pre-sorted data. Randomized Quicksort solves this cleanly by avoiding predictable pivot choices, making it far more reliable across all kinds of inputs.

Comparison of Deterministic and Randomized Quicksort

Feature	Deterministic Quicksort	Randomized Quicksort
Pivot Selection	Last element (fixed pivot)	Random element
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	$O(n^2)$	$O(n^2)$ (extremely unlikely)
Space Complexity	O(log n) average, O(n) worst case	O(log n) average, O(n) worst case
Performance on Sorted Data	Poor (can approach $O(n^2)$)	Excellent (avoids predictable bad pivots)
Performance on Random Data	Good	Good
Chance of Worst Case	Higher for specific input patterns	Extremely low due to random pivot selection

In this assignment, we implemented, analyzed, and compared the performance of both deterministic and randomized versions of the Quicksort algorithm. The deterministic version works well on average with an O(n log n) time complexity, but it heavily suffers and drops to O(n^2) whenever bad pivot picks lead to totally unbalanced partitions. Randomized Quicksort fixes this issue by choosing pivots at random, which keeps the expected runtime at O(n log n) regardless of how the input data is arranged.

Our practical testing backed up the theoretical analysis completely. Both implementations performed almost identically on random datasets since the splits naturally stayed balanced. However, on sorted and reverse-sorted inputs, the randomized version was the clear winner it easily avoided the predictable bad pivots that bogged down the deterministic code.

Overall, the project clearly shows why randomized Quicksort is the go-to approach when we don't know the nature of our input data beforehand. Its ability to stay consistently fast while effectively dodging worst-case scenarios makes it one of the most practical and reliable sorting algorithms out there.

References:

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
2. Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
3. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Python. Wiley.